

DESARROLLO ORIENTADO A OBJETOS [DOPO]

2026-2

Ciclo 1

Materia: Desarrollo orientado a objetos

Grupo: 7

Integrantes

Jhon Eduard Duran Ceballos

ID: 1000100325

Correo: jhon.duran-c@mail.escuelaing.edu.co

Andres Steven Oyuela Mendez

ID: 1000109048

Correo: andres.oyuela-m@mail.escuelaing.edu.co

Introducción

El objetivo es diseñar e implementar un simulador de máquina tragamonedas que aplique rigurosamente los principios de POO: encapsulamiento, cohesión, bajo acoplamiento y extensibilidad. Utilizando como base el framework de clases shapes, el sistema se construye de manera incremental a través de 9 mini-ciclos de desarrollo. Este enfoque metodológico asegura que cada componente (desde la gestión de ruedas y símbolos hasta la detección de Jackpot y la visibilidad del simulador) sea validado, documentado y probado, culminando en una entrega que cumple con estándares de diseño en Astah, control de versiones en Git y una retrospectiva técnica detallada.

Métodos de uso

1. **SlotMachine()**: Inicializa la máquina tragamonedas con sus componentes básicos y el lienzo (Canvas).
2. **makeVisible()**: Hace visible la interfaz gráfica del simulador en el tablero.
3. **makeInvisible()**: Oculta la interfaz gráfica del simulador, permitiendo que la lógica siga funcionando en segundo plano.
4. **addWheel(int index)**: Añade una nueva rueda a la máquina en la posición especificada.
5. **delWheel(int index)**: Elimina una rueda existente de la máquina en la posición especificada.
6. **addSymbol(String color, String shape)**: Añade un nuevo símbolo (definido por color y forma) a la colección disponible.
7. **delSymbol(String color, String shape)**: Elimina un símbolo específico de la colección disponible.
8. **configuration()**: Retorna o muestra la configuración actual de las ruedas y sus símbolos.
9. **distinctSymbols()**: Verifica la cantidad de símbolos distintos presentes en la máquina.
10. **isJackpot()**: Determina si la configuración actual de las ruedas es ganadora (todos los símbolos coinciden).
11. **ok()**: Valida si el estado actual de la máquina es consistente y operativo.
12. **symbols()**: Lista todos los símbolos o ruedas actualmente registrados en el sistema.
13. **spin()**: Gira todas las ruedas de la máquina, actualizando los símbolos visibles.

14. **spin(int index)**: Gira una rueda específica indicada por su índice, actualizando su símbolo visible.
15. **placeSymbol(int index, String color)**: Asigna un símbolo específico (por su color) a la rueda indicada, permitiendo manipular el estado del juego.

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

MINI-CICLO 0 — Entender y preparar el proyecto

Estado: Completo

Reflexion: Análisis de la jerarquía de 'shapes' para definir una arquitectura orientada a objetos extensibles.

Logro: Arquitectura definida y roles repartidos con éxito.

- Checklist:

- ☒ Crear el proyecto BlueJ con nombre *slotMachine*.
- ☒ Revisar completamente las clases de *shapes*.
- ☒ Identificar qué funcionalidades tienen *Canvas*, *Circle*, *Triangle* y *Rectangle*.
- ☒ Identificar qué clases pueden reutilizarse directamente.
- ☒ Identificar qué clases necesitan extenderse.
- ☒ Definir las clases nuevas de la máquina tragamonedas.
- ☒ Definir responsabilidades de cada clase.
- ☒ Elaborar un primer bosquejo del diagrama de clases.
- ☒ Definir los mini-ciclos y repartir responsabilidades del equipo.

MINI-CICLO 1 — Crear la máquina tragamonedas

Estado: Completo

Reflexión: Instanciación de 'SlotMachine' delegando correctamente las responsabilidades gráficas al 'Canvas'.

Logro: Máquina visualizable en el tablero con constructores documentados.

- Checklist:

- ☒ Crear la clase principal *SlotMachine*.
- ☒ Definir sus atributos principales.
- ☒ Definir constructor(es).
- ☒ Crear la representación visual básica.
- ☒ Crear el *Canvas*.

- ☒ Mostrar la máquina en el tablero.
- ☒ Probar la creación de una máquina.
- ☒ Documentar los métodos.

MINI-CICLO 2 — Crear y administrar las ruedas

Estado: Completo

Reflexión: Aplicación de encapsulamiento para gestionar la adición y eliminación dinámica de ruedas.

Logro: Gestión de ruedas implementada con validaciones mediante `JOptionPane`.

- Checklist:

- ☒ Crear la representación de una rueda.
- ☒ Determinar los atributos de una rueda.
- ☒ Crear las ruedas inicialmente.
- ☒ Adicionar una rueda.
- ☒ Eliminar una rueda.
- ☒ Actualizar la representación gráfica.
- ☒ Validar que no se eliminen ruedas cuando la operación no sea válida.
- ☒ Mostrar *JOptionPane* cuando corresponda.

MINI-CICLO 3 — Crear y administrar los símbolos

Estado: Completo

Reflexión: Diseño de la relación entre Símbolo y Rueda asegurando colores distintos.

Logro: Sistema de símbolos flexible, extensible y correctamente asociado.

- Checklist:

- ☒ Crear la representación de un símbolo.
- ☒ Asociar un símbolo con una representación gráfica.
- ☒ Permitir adicionar símbolos.
- ☒ Permitir eliminar símbolos.
- ☒ Asignar colores diferentes.
- ☒ Validar operaciones incorrectas.
- ☒ Actualizar las ruedas cuando cambien los símbolos.

MINI-CICLO 4 — Integrar ruedas y símbolos

Estado: Completo

Reflexión: Sincronización entre el estado del modelo y la representación visual de los símbolos.

Logro: Integración funcional lograda manteniendo bajo acoplamiento.

- Checklist:

- ☒ Asociar símbolos con las ruedas.
- ☒ Definir qué símbolo está visible en cada rueda.
- ☒ Representar correctamente el símbolo actual.
- ☒ Actualizar visualmente la máquina.
- ☒ Mantener sincronizados modelo y representación gráfica.
- ☒ Probar diferentes cantidades de ruedas y símbolos.

MINI-CICLO 5 — Girar las ruedas

Estado: Completo

Reflexión: Lógica de giro basada en cambios de estado internos y actualización gráfica inmediata.

Logro: Giro de ruedas validado funcionalmente con control de parámetros.

- Checklist:

- ☒ Crear método para girar una rueda.
- ☒ Crear método para girar las ruedas.
- ☒ Determinar cómo cambia el símbolo visible.
- ☒ Actualizar gráficamente la rueda.
- ☒ Validar índices o parámetros incorrectos.
- ☒ Probar diferentes giros.
- ☒ Verificar que el estado interno coincida con lo mostrado.

MINI-CICLO 6 — Consultar los símbolos

Estado: Completo

Reflexión: Diseño de métodos de consulta que garantizan la integridad sin exponer atributos internos.

Logro: Consulta pública segura del estado actual de los símbolos.

- Checklist:

- ☒ Consultar los símbolos visibles.
- ☒ Crear métodos de consulta.
- ☒ Definir qué información debe retornar cada método.
- ☒ Evitar exponer directamente los atributos internos.
- ☒ Probar consultas después de diferentes giros.

MINI-CICLO 7 — Detectar el estado ganador

Estado: Completo

Reflexión: Implementación de la detección lógica de Jackpot comparando el estado de todas las ruedas.

Logro: Cambio visual dinámico del simulador al detectar la configuración ganadora.

- Checklist:

- ☒ Crear método que determine si la configuración es ganadora.
- ☒ Comparar los símbolos visibles.
- ☒ Probar configuraciones ganadoras.
- ☒ Probar configuraciones no ganadoras.
- ☒ Cambia visualmente la máquina cuando gana.

MINI-CICLO 8 — Visibilidad del simulador

Estado: Completo

Reflexión: Manejo del estado de visibilidad preservando la funcionalidad del simulador en modo oculto.

Logro: Simulador totalmente operativo en ambos estados de visibilidad.

- Checklist:

- ☒ Crear método para hacer visible el simulador.
- ☒ Crear método para hacerlo invisible.
- ☒ Comprobar comportamiento visual.
- ☒ Determinar cuándo deben aparecer mensajes.
- ☒ Verificar funcionamiento en modo invisible.
- ☒ Probar todas las funcionalidades en ambos modos.

MINI-CICLO 9 — Terminar y validar el simulador

Estado: Completo

Reflexión: Refactorización y revisión de documentación para asegurar la calidad del código.

Logro: Sistema estabilizado, documentado bajo estándares Java y sin métodos excesivamente extensos.

- Checklist:

- ☒ Implementar terminar el simulador.
- ☒ Revisar el estado de *Canvas*.
- ☒ Verificar que no queden operaciones inválidas.
- ☒ Probar todas las funcionalidades.
- ☒ Revisar errores.
- ☒ Revisar documentación Java.

- ☒ Revisar nombres de clases, métodos y atributos.
- ☒ Verificar encapsulamiento.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

El estado actual es 100% completado. Hemos abarcado absolutamente todos los mini-ciclos de desarrollo. El simulador cumple con la creación de la máquina, gestión de ruedas y símbolos, lógicas de giro, validación del premio (Jackpot) y las reglas de visibilidad exigidas por el diagrama UML. Además, cumplimos con todos los requisitos de usabilidad, incluyendo los mensajes de error a través de JOptionPane.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Invertimos aproximadamente 10 a 12 horas/hombre. Este tiempo incluye la comprensión de la librería gráfica preexistente, la codificación de las clases SlotMachine, Wheel, Symbol y Lever, las pruebas iterativas en BlueJ y la corrección de errores visuales (renderizado de capas).

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue implementar un sistema de capas visuales y animaciones fluidas respetando el encapsulamiento orientado a objetos. Logramos crear el efecto de "borde negro" dinámico, integrar una base responsiva y animar la palanca (Lever), asegurando que las casillas siempre se dibujaran por encima del fondo (gris o dorado) utilizando el método de redibujado, todo esto sin romper la estructura exigida en el UML.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor reto técnico estuvo relacionado con el motor de renderizado de la clase Canvas.

- Problema: Al intentar cambiar el color de la máquina al ganar el Jackpot o al actualizar las ruedas, las figuras traseras saltaban al frente, tapando por completo los símbolos. Además, presenciamos un flickering (parpadeo)

debido al método `redraw()` del Canvas, el cual borra toda la pantalla antes de pintar.

- Solución: Resolvimos el problema de las capas implementando métodos estratégicos como `actualizarRuedas()` y `redibujar()` en nuestras clases, los cuales forzaban a Java a pintar primero los fondos y la base, y en última instancia las ruedas y sus símbolos, garantizando el orden correcto en el eje.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Hicimos un excelente trabajo prestando atención a los detalles de usabilidad (diseño visual, bordes, animaciones) y probando exhaustivamente la máquina en escenarios de error (por ejemplo, validar símbolos repetidos en modo visible). Nos comprometemos a realizar un análisis matemático más detallado de las coordenadas gráficas desde el inicio, para evitar errores de posicionamiento espacial (como cuando dibujamos la base fuera de los límites de los 400 píxeles de alto del Canvas).

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

La práctica de Entregas Pequeñas (Small Releases / Mini-ciclos) fue fundamental. Dado que estábamos trabajando con una librería gráfica donde un error de coordenadas rompe la experiencia visual, codificar y probar funcionalidad por funcionalidad (ej. primero lograr que una rueda se vea, luego que se mueva, luego que gire) impidió que el código se volviera inmanejable.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Escuela Colombiana de Ingeniería. (2026). Desarrollo Orientado Por Objetos [DOPO-POOB] Proyecto Inicial Ciclo No 1. Documento de requisitos y diagrama UML. (Esta fue la más útil, ya que dictó la arquitectura estricta del proyecto).
- Quig, B., & Kolling, M. (n.d.). Canvas Class Documentation (Version 1.6). Librería gráfica BlueJ Shapes Demo.
- ICPC. (2025). Problem I: Slot Machine. Maratón de Programación Internacional.